

Containerizing Laravel with Docker

Run Module 2's Bearer Token API in three containers.

Learning Goals

- Describe how Nginx, PHP-FPM, and MySQL work together as three containers.
- Compare the Docker setup with Module 2's local installation.
- Explain why the database survives `docker compose down` (named volume vs. bind mount).

The Goal

In Module 2 you installed PHP, Composer, and MySQL on your computer and ran `run1-5.sh` .

Now one script builds the **same project** inside Docker:

	Module 2	This lesson
Install on your computer	PHP, Composer, MySQL	Docker only
Web server	<code>php artisan serve</code>	Nginx container
Database	local MySQL, <code>sudo mysql</code> setup	MySQL container
Script	<code>run1-5.sh</code>	<code>run1-5-docker.sh</code>

Three Containers

Browser :8000 → Nginx :80 → PHP-FPM :9000 → MySQL :3306

Service	Image	Role
nginx	nginx:alpine	Receives requests, serves public/
php	php:8.4-fpm	Runs Laravel (Laravel 13 needs PHP 8.3+)
mysql	mysql:8.0	Stores studentdb

Nginx never runs PHP. It forwards .php requests to PHP-FPM.

Same Pattern as the Previous Example

	simple-php-nginx	Laravel example
Shared code	<code>./www</code> in both	<code>./student-api</code> in both
Nginx → PHP	<code>php-app:9000</code>	<code>php:9000</code>
Nginx root	<code>/var/www/html</code>	<code>/var/www/html/public</code>
Routing	<code>=404</code>	<code>/index.php?</code> <code>\$query_string</code>
Database	none	MySQL + volume

Same idea: one folder, mounted into the Nginx and PHP

Port 8000 Is Shared

The previous PHP + Nginx example also uses port 8000.

- Stop it first: `docker compose down` in `simple-php-nginx`.
- Or change the left side of `"8000:80"` in `docker-compose.yml`
(for example `"8001:80"`) and open that port instead.

Only one container can own a host port at a time.

Same Names as Module 2

Module 2	Docker folder
<code>run1-5.sh</code>	<code>run1-5-docker.sh</code>
<code>student-api1-5/</code> (lesson files)	<code>student-api1-5/</code> (identical copy)
<code>student-api/</code> (created project)	<code>student-api/</code>
<code>test1-5.sh</code>	<code>test1-5-docker.sh</code>
<code>run1-5-tests/</code>	<code>run1-5-tests/</code>

The new files are the Docker setup: `docker-compose.yml` ,
`docker/` ,
`.env` , and `.env.docker` .

Folder Layout

4. Containerizing Laravel with Docker/

— docker-compose.yml	three services
— docker/	one folder per service
— nginx/	(Dockerfile, nginx.conf)
— php/	(Dockerfile)
— mysql/	(Dockerfile, init.sql)
— .env	read by Docker Compose
— .env.docker	becomes student-api/.env
— student-api1-5/	lesson files (from Module 2)
— run1-5-docker.sh	build everything
— test1-5-docker.sh	test the API
— student-api/	created by the script

docker-compose.yml: Nginx

```
services:
  # NGINX Web Server
  nginx:
    build: ./docker/nginx
    ports:
      - "8000:80"    # Same host port as the previous PHP + NGINX example: stop that one first
    volumes:
      - ./student-api:/var/www/html
    depends_on:
      - php
```

- `ports` : host port 8000 → container port 80.
- `volumes` : the project folder appears inside the container.
- `depends_on` : start `php` before `nginx` .

docker-compose.yml: PHP and MySQL

```
php:
  build: ./docker/php
  volumes:
    - ./student-api:/var/www/html
  depends_on:
    mysql:
      condition: service_healthy
```

- `build: ./docker/php` tells Compose to build the PHP image from `./docker/php/Dockerfile` and use it to run the `php` service.
- `php` mounts the **same** `./student-api` folder as `nginx`.
- `service_healthy`: wait until MySQL answers its health check, not just until it has started.
- MySQL data lives in the named volume `mysql-data`, so it

Two Environment Files

File in the template	Read by	Purpose
<code>.env</code>	Docker Compose	Initialize MySQL
<code>.env.docker</code>	Copied to <code>student-api/.env</code>	Connect Laravel to MySQL

Both are hidden files.

- The script uses them but does not create them: Compose reads `.env`, and the script copies `.env.docker` to `student-api/.env`.
- When making a working copy, use `cp -r folder my-project ; folder/*` skips hidden files.

Database Settings in Both Files

`.env` (MySQL creates these on a new volume):

```
MYSQL_DATABASE=studentdb  
MYSQL_USER=ase230  
MYSQL_PASSWORD=ase230pass  
MYSQL_ROOT_PASSWORD=root_password
```

`.env.docker` (Laravel connects with these):

```
DB_CONNECTION=mysql  
DB_HOST=mysql  
DB_PORT=3306  
DB_DATABASE=studentdb  
DB_USERNAME=ase230  
DB_PASSWORD=ase230pass
```

- `DB_HOST=mysql` is the service name. `127.0.0.1` would mean the PHP container itself.
- Same database and account as Module 1 and 2.
- No `CREATE DATABASE` or `GRANT` commands are needed: MySQL creates the database and user from `MYSQL_*` on first initialization.
- Changing `MYSQL_*` later does not update an existing MySQL account.

Containers (Volumes) vs. Data

The named Docker volume `mysql-data` stores MySQL's database files separately from the container.

- Compose mounts it at `/var/lib/mysql` inside the MySQL container:

```
Docker volume mysql-data → MySQL container: /var/lib/mysql
```

The database lives in this volume, not in the container's writable layer or in `student-api/` .

Command	Containers	Database data
<code>docker compose down</code>	Removed	Kept (<code>mysql-data</code> volume)
<code>docker compose down -v</code>	Removed	Deleted
<code>docker compose up -d --wait</code>	Created and started	Reused

- Deleting `student-api/` does **not** delete the database; The next run finds old tables (see `4-2`).
- To start completely over: `docker compose down -v` , delete `student-api/` , run the script again

Bind Mount and Permissions

`student-api/` is mounted over `/var/www/html`, so a `chown` done while building the image is hidden.

- PHP-FPM runs as `www-data` and must write to `storage/` and `bootstrap/cache/`.
- On Linux/WSL2 a host-owned folder blocks it: "Permission denied".
- The script runs `chown -R www-data:www-data storage bootstrap/cache` inside the container.

This changes the owner of those host folders. Fine for local practice.

What Is New in the Docker Files

Lesson 3 used the official Nginx image, a small PHP Dockerfile, and two services.

- This example needs more, so every service has its own Dockerfile.

	Lesson 3	This lesson
Nginx	<code>image: nginx:alpine</code> + mounted <code>default.conf</code>	Own Dockerfile + complete <code>nginx.conf</code>
PHP	<code>pdo</code> , <code>pdo_mysql</code>	Laravel extensions and Composer
Database	none	MySQL with a

Nginx for Laravel

The Nginx container now has its **own Dockerfile**, and the configuration is much longer.

Config Inside the Image

```
nginx:  
  build: ./docker/nginx
```

```
FROM nginx:alpine  
  
# Copy NGINX configuration  
COPY nginx.conf /etc/nginx/nginx.conf
```

- The previous **Docker with PHP and NGINX** example mounted `default.conf` into the running container.
- Here `COPY` puts the config **into the image** when it is built.
- `build: ./docker/nginx` uses that folder as the build context, so `COPY nginx.conf` finds `docker/nginx/nginx.conf`.

A Complete nginx.conf

The previous **Docker with PHP and NGINX** example replaced only `conf.d/default.conf`.

- This Dockerfile replaces the **whole** `/etc/nginx/nginx.conf`, so it must contain the outer blocks:

```
events {  
    worker_connections 1024;  
}  
  
http {  
    include /etc/nginx/mime.types;  
    ...  
    gzip on;  
    ...  
    server {  
        listen 80;  
        server_name localhost;  
        root /var/www/html/public;  
    }  
}
```

- `events` and `http` are required in a complete file.
- `mime.types` sets the `Content-Type` of files; `gzip on` compresses responses.
- `server` is the part you already know from lesson 3.

Laravel Routing: root and try_files

```
root /var/www/html/public;  
...  
location / {  
    try_files $uri $uri/ /index.php?$query_string;  
}
```

- `root` is `public/`, not the project folder. `.env`, `app/`, and `vendor/` stay **outside** the web root and cannot be downloaded.
- In the earlier PHP + NGINX example, NGINX returned `404` when no file matched the request.
- Here, NGINX sends unmatched requests to Laravel's `index.php`. Laravel checks its routes and returns the response, or a `404` if no route matches.

These are examples:

Request	Nginx does
<code>/favicon.ico</code> (file exists)	Serves the file
<code>/api/login</code> (no such file)	Runs <code>/index.php?...</code> → Laravel route
<code>/api/students/1</code>	Runs <code>/index.php?...</code> → Laravel route

`$query_string` keeps the query part, such as `?name=John` .

Passing PHP to PHP-FPM

```
location ~ /\.php$ {  
    fastcgi_pass php:9000;  
    fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;  
    include fastcgi_params;  
    fastcgi_hide_header X-Powered-By;
```

- `php:9000` : the Compose **service name** is now `php` . In lesson 3 it was `php-app` .
- `$realpath_root` is the real path of the root folder. Lesson 3 used `$document_root` .
- `fastcgi_hide_header X-Powered-By` removes the header that shows the PHP version.

Static Files and Security

Setting	What it does
<code>expires 1y</code> and <code>Cache-Control "public, immutable"</code>	The browser keeps CSS, JavaScript, and images for a year
<code>add_header X-Frame-Options "SAMEORIGIN" always</code>	Other sites cannot put your page in a frame
<code>add_header X-Content-Type-Options "nosniff" always</code>	The browser must not guess the file type
<code>`location ~* .(env</code>	git
<code>location ~ /\. { deny all; }</code>	Never serve hidden files (lesson 3 had this rule too)

PHP for Laravel

The earlier `simple-php-nginx` example installed `pdo` and `pdo_mysql`.

- Laravel 13 needs PHP 8.3 or newer, additional PHP extensions, and Composer, so this image uses PHP 8.4 and installs those requirements.

System Packages

```
# Install system dependencies
RUN apt-get update && apt-get install -y \
    git \
    curl \
    libpng-dev \
    libonig-dev \
    libxml2-dev \
    libzip-dev \
    zip \
    unzip \
    libfreetype6-dev \
    libjpeg62-turbo-dev \
    && rm -rf /var/lib/apt/lists/*
```

- `git`, `zip`, and `unzip` : Composer uses them to download packages.
- `lib...-dev` : libraries that PHP needs to **compile** the extensions on the next slide.
- `rm -rf /var/lib/apt/lists/*` deletes the package lists, so the image stays smaller.

PHP Extensions

```
# Install PHP extensions required by Laravel
RUN docker-php-ext-configure gd --with-freetype --with-jpeg \
    && docker-php-ext-install \
        pdo_mysql \
        mbstring \
        exif \
        pcntl \
        bcmath \
        gd \
        zip
```

- `pdo_mysql` : talk to MySQL. `mbstring` : multibyte strings.
`bcmath` : exact decimal math.
- `gd` : images (configured with FreeType and JPEG). `zip` :
read zip files.
- `docker-php-ext-install` is the same helper that lesson 3
used for `pdo` .

Composer in the PHP Image

Composer manages PHP project dependencies.

- Because Composer runs with PHP, the PHP image needs both PHP and Composer.

```
# Install Composer  
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
```


- `composer:2` means the `composer` image with tag `2` on Docker Hub.
 - During the build, Docker downloads it if it is not already on the computer.
- `COPY --from=composer:2` copies the Composer executable from the Composer image into this PHP image; it does not start another container.
- `run1-5-docker.sh` creates temporary containers from the PHP image to run `composer create-project` and `composer install`.
- The script therefore needs Docker, not PHP or Composer installed on the host computer.

HEALTHCHECK

A health check is a command that Docker runs inside the container again and again.

```
# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD php-fpm -t || exit 1
```

Option	Meaning
<code>--interval=30s</code>	Run the check every 30 seconds
<code>--timeout=3s</code>	The check fails if it takes longer
<code>--start-period=5s</code>	Failures at startup are not counted
<code>--retries=3</code>	Three failures in a row mean unhealthy

- Exit code `0` is healthy; anything else is a failure.
- `php-fpm -t` tests the PHP-FPM configuration.
- Nginx uses `wget` (`nginx:alpine` has no `curl`). MySQL uses `mysqladmin ping`.
- `docker compose ps` shows `(healthy)`, and `up --wait` waits for it.

MySQL and the Network

The database is the new service, and the services now share a named network.

The MySQL Service

```
# MySQL Database (Custom Build)
mysql:
  build: ./docker/mysql          # Use custom Dockerfile instead of official image
  environment:
    - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
    - MYSQL_DATABASE=${MYSQL_DATABASE}
    - MYSQL_USER=${MYSQL_USER}
    - MYSQL_PASSWORD=${MYSQL_PASSWORD}
  volumes:
    - mysql-data:/var/lib/mysql
```

- `environment` sets variables **inside** the container. `${...}` takes the value from the `.env` file next to `docker-compose.yml`.
- The MySQL image reads `MYSQL_*` the first time it starts and creates the database and user.

- `mysql-data:/var/lib/mysql` is a **named volume**: Docker stores the database files, not a folder of yours.
- It is declared at the end of the compose file:

```
volumes:  
  mysql-data:
```

- `./student-api:/var/www/html` is a bind mount (a folder of yours). `mysql-data` is a volume (managed by Docker).

The MySQL Dockerfile

```
FROM mysql:8.0

# Allow remote connections (for Docker networking)
ENV MYSQL_ROOT_HOST=%

# Minimal configuration for MySQL 8.0 compatibility
RUN echo '[mysqld]' > /etc/mysql/conf.d/laravel.cnf \
    && echo 'default-authentication-plugin=mysql_native_password' >> /etc/mysql/conf.d/laravel.cnf

EXPOSE 3306
```

- `MYSQL_ROOT_HOST=%` : root may log in from other containers, not only from `localhost` .
- The `RUN` lines write a small config file that keeps the older `mysql_native_password` login method.
- `EXPOSE 3306` documents the port. It is **not** published to your computer, so only the other containers can reach MySQL.

The MySQL Health Check

```
healthcheck:
  test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -u root -p$$MYSQL_ROOT_PASSWORD --silent"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 30s
```

- Same options as `HEALTHCHECK`, but written in the compose file.
- `$$` is a literal `$`: the **container** reads `MYSQL_ROOT_PASSWORD`, not Compose.
- `start_period: 30s`: MySQL needs time to initialize a new volume.
- `php` waits for this check (`service_healthy`).

The Named Network

```
networks:  
  laravel-network:  
    driver: bridge
```

```
networks:  
  - laravel-network
```

- Before, we used the default network that Compose creates. Here we declare one and attach every service.
- On a network, a service name is a host name: `php:9000` in `nginx.conf`, `DB_HOST=mysql` in `.env.docker`.
- Only `nginx` publishes a port (`8000:80`). `php` and `mysql` are reachable **only inside** this network.

Service Names, Not Container Names

Before, we set `container_name: web-server` and `php-container`. This compose file does not.

- Docker generates unique names, so two copies of the project do not collide.
- You do not need the container name. Use the **service** name:

```
docker compose exec php php -v  
docker compose logs mysql
```

Make a Change

You changed	Run
<code>docker/nginx/nginx.conf</code>	<code>docker compose up -d --build nginx</code> (the config is inside the image)
<code>docker/php/Dockerfile</code>	<code>docker compose up -d --build php</code>
Laravel code in <code>student-api/</code>	Nothing: Nginx and PHP read the mounted folder
<code>student-api/.env</code>	<code>docker compose exec php php artisan config:clear</code>
<code>MYSQL_*</code> in <code>.env</code>	<code>docker compose down -v</code> , then start again (new volume)

Command Summary

Students should remember:

Command	Meaning
<code>docker compose build</code>	Build the three images
<code>docker compose up -d --build nginx</code>	Rebuild one service after changing its Dockerfile or config
<code>docker compose up -d --wait</code>	Start all services; wait until healthy
<code>docker compose ps</code>	List services and their health
<code>docker compose logs php mysql</code>	Read the logs